

# Escopo Calibrado para a Mitigação de DDoS de Camada de Aplicação com um Grafo de Conhecimento Centrado na Sessão

Natanael Oliveira, Arthur Kobielski, Marcos Luna, Luigi Carro, Jéferson Campos Nobre e Eder John Scheid  
Instituto de Informática, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre, Brasil  
{natanael.oliveira, arthur.kobielski, marcos.luna, carro, jcnobre, ejscheid}@inf.ufrgs.br

**Resumo**—Numa campanha distribuída de Slow HTTP DoS, cada origem fica abaixo dos limiares por origem, e a mitigação passa a depender do escopo: quais clientes bloquear, poupando os usuários legítimos. Sobre um grafo de conhecimento centrado na sessão, um teste binomial de enriquecimento só bloqueia uma impressão digital TLS quando a fração das origens do alarme que a apresentam seria improvável no tráfego normal, num nível calibrado para um orçamento de falsos alarmes. Em tráfego gerado, o escopo natural, a impressão digital mais comum do agrupamento, bloqueia 0% de uma *botnet* espalhada por cinco ou mais pilhas TLS e atinge 39% do tráfego legítimo. O teste bloqueia 90% da *botnet* com até 25 pilhas, sem dano colateral observado. Um filtro de impressões inéditas faz o mesmo, com 0,03% de dano colateral. Nas pilhas que clientes reais compartilham, o teste bloqueia 85% dos atacantes, com 2% de dano colateral, enquanto esse filtro não bloqueia nenhum. Um modelo treinado com atributos entre sessões (AUC de 0,98) cai para 0,48 a 0,75 quando testado numa *botnet* com outro número de pilhas. Em oito dias de tráfego de quatro *endpoints* de uma CDN, frotas de clientes que se ativam ao mesmo tempo impõem um piso de calibração. A menor *botnet* que o teste binomial consegue apontar tem 16% da janela do *endpoint* mais movimentado e de 4 a 12 janelas inteiras nos *endpoints* pequenos. Uma beta-binomial que modela as frotas rebaixa esse piso para 3% e para 0,9 a 2,8 janelas, no ponto de operação de um *z-score* calibrado para o mesmo orçamento, com falsos alarmes em 0,4% das janelas limpas. Todo escopo calibrado bloqueia de 67 a 71% de uma *botnet* do tamanho da janela típica do *endpoint* mais movimentado. Num dia novo, analisado como planejado de antemão, a configuração binomial gera 2 falsos alarmes em 1 152 janelas. A ontologia é compilada numa consulta que reproduz as contagens de origens e de pares no mesmo /24 exportadas pela operadora em dois dias de produção, e o escopo é exportado em STIX 2.1.

**Index Terms**—gestão de segurança, grafo de conhecimento, DDoS de camada de aplicação, modelos semânticos, raciocínio automatizado, explicabilidade, política de mitigação

## I. INTRODUÇÃO

Os ataques de negação de serviço distribuída (DDoS) na camada de aplicação continuam crescendo em volume e em frequência. Em agosto de 2023, o HTTP/2 Rapid Reset (CVE-2023-44487) elevou os picos mitigados a 398 milhões de requisições por segundo [1], [2]. Um desses ataques chegou a 201 milhões a partir de cerca de vinte mil máquinas [3], e, no segundo trimestre de 2025, a Cloudflare bloqueou mais de

6 500 ataques hipervolumétricos [4]. As defesas volumétricas absorvem ataques dessa magnitude. A detecção fica mais difícil na outra ponta do espectro, a de baixa taxa, onde está o *Slow HTTP DoS distribuído*. O Slowloris e seus descendentes, além de variantes baseadas em taxa que também mantêm conexões abertas, como HULK e GoldenEye [5], esgotam um servidor a partir de várias origens coordenadas. Cada origem envia requisições pequenas o bastante para não ultrapassar nenhum limiar por origem.

Além disso, uma campanha furtiva molda cada sessão para parecer legítima nos sinais por sessão em que os detectores se apoiam, como a duração da conexão e a entropia das rotas acessadas [6]. O que continua a distinguir o ataque está na estrutura *entre* as sessões: a mesma impressão digital de cliente aparece em muitas origens, e essas origens convergem para um mesmo *endpoint*. Visto assim, o problema é tanto de gestão de redes e serviços quanto de detecção, porque agir exige uma decisão: *quais* clientes tratar, com base em *qual* evidência, e com um filtro estreito o bastante para poupar os usuários legítimos do serviço atacado.

Os trabalhos existentes tratam do lado da detecção e deixam em aberto o lado operacional. Uma meta-análise de 75 estudos mostra que 47% dos detectores de DDoS de camada de aplicação extraem seus atributos de sessões HTTP [7]. Essas sessões são reduzidas a um vetor numérico antes de chegarem ao classificador, que então não consegue raciocinar sobre impressões digitais que se repetem em várias origens, nem sobre padrões que só existem entre sessões individualmente normais. Detectores baseados em grafos preservam essas relações: o KLAGÉ [8] classifica os nós de grafos de conhecimento construídos a partir de *logs* de rede e gera relatórios em linguagem natural. Três oportunidades continuam em aberto. Em primeiro lugar, a unidade de raciocínio é o *nó de rede* (porta, IP, fluxo), enquanto a coordenação na camada 7 só aparece no nível da sessão de aplicação. Em segundo lugar, a coordenação fica implícita em *embeddings* aprendidos, de modo que nenhum operador consegue inspecionar as relações que levaram ao veredicto. Por fim, a saída se limita a um relatório, e agir sobre ele exige uma política de mitigação cujo escopo decorra do grafo.

Tabela I  
COMPARAÇÃO DOS TRABALHOS RELACIONADOS

| Trabalho             | Ano  | Objetivo               | Unidade de Raciocínio | Relação entre Sessões          | Explicação do Veredicto | Escopo de Mitigação | Colateral     |
|----------------------|------|------------------------|-----------------------|--------------------------------|-------------------------|---------------------|---------------|
| [9]                  | 2012 | DDoS L7                | Sessão como Vetor     | -                              | -                       | -                   | -             |
| [6]                  | 2023 | DoS L7                 | Sessão como Vetor     | -                              | -                       | -                   | -             |
| [10]                 | 2024 | Tráfego de <i>Bots</i> | Impressão Digital JA4 | Proprietária                   | -                       | Não divulgado       | Não divulgado |
| [8]                  | 2026 | Ameaças de Rede        | Nó de Rede            | Implícita (Aprendida)          | A posteriori (LIME/LLM) | -                   | -             |
| <b>Este Trabalho</b> | 2026 | DDoS L7 Distribuído    | Sessão de Aplicação   | Explícita (Tipada e Ponderada) | Derivação               | Derivado            | Reportado     |

Este artigo as aborda ao tratar o escopo da mitigação como um teste estatístico calibrado, executado sobre um grafo de conhecimento de sessões HTTP. Cada requisição HTTP/TLS é associada a um nó `ApplicationSession` de um grafo OWL mantido numa janela deslizante, e as sessões são ligadas por sub-propriedades, cada uma ponderada pelo custo que o atacante teria para contornar o sinal correspondente. Uma regra SPARQL/SWRL dispara o alarme, e o escopo desse alarme é o conjunto de impressões digitais que um teste binomial de enriquecimento considera sobrerrepresentadas em relação ao tráfego normal do *endpoint*, num nível calibrado para um orçamento de falsos alarmes. O alvo são campanhas conduzidas por *botnets* de dispositivos comprometidos, cujo operador não consegue trocar as pilhas TLS a baixo custo. *Bots* que apresentam a impressão digital de um navegador são avaliados como a fronteira do método. As contribuições são quatro. (i) Mostramos que o escopo natural, a propriedade compartilhada pela maior parte do agrupamento do alarme, bloqueia usuários legítimos sem atingir nenhum atacante assim que a *botnet* se espalha por algumas pilhas TLS. O escopo por enriquecimento, tendo como referência um perfil livre de ataque, bloqueia o ataque, inclusive nas pilhas que clientes reais compartilham, que um filtro de impressões inéditas deixa passar. Esse escopo dispensa rótulos, enquanto um modelo treinado com atributos entre sessões falha numa *botnet* com outro número de pilhas. (ii) Em oito dias de tráfego de uma CDN, medimos até onde o escopo calibrado alcança. Frotas legítimas impõem um piso ao tamanho da *botnet* que o teste consegue apontar, e uma distribuição de referência beta-binomial que modela essas frotas torna esse piso várias vezes menor. No regime furtivo, o escopo aponta a *botnet* que o gatilho de volume, em sua maior parte, deixa passar. Comparamos o teste com linhas de base calibradas para o mesmo orçamento de falsos alarmes e conferimos a taxa de falsos alarmes do teste num dia novo, analisado com a configuração fixada de antemão. (iii) Especificamos o grafo de sessões como uma ontologia OWL com *seis sub-propriedades tipadas*, três delas usadas na avaliação, a partir da qual a consulta de contagens é compilada. Em dois dias de produção, essa consulta reproduz janela a janela os agregados da operadora, e o escopo é exportado com sua cadeia de evidência em STIX 2.1. (iv) Reportamos a taxa de falsos positivos do filtro resultante como dano colateral sobre o tráfego legítimo, em janelas limpas e em janelas de ataque.

A Seção II revisa os trabalhos relacionados, a Seção III apresenta a abordagem e a Seção IV, a metodologia de avaliação. A Seção V reporta os resultados, a Seção VI discute o

uso operacional da abordagem e suas limitações, e a Seção VII conclui.

## II. TRABALHOS RELACIONADOS

A Tabela I compara este trabalho com o estado da arte em quatro dimensões: a unidade de raciocínio, a forma como as relações entre sessões são representadas, o tipo de explicação oferecida e se um escopo de mitigação e o seu custo decorrem dela.

### A. Grafos de Conhecimento em Segurança e Gestão

A primeira geração de grafos de conhecimento em segurança era *estática*: entidades extraídas de registros Common Vulnerabilities and Exposures (CVE), de relatórios e de *blogs* eram normalizadas e ligadas numa base consultável [11], [12]. O conhecimento defensivo segue o mesmo padrão no D3FEND, um grafo OWL de técnicas de contramedida e dos artefatos sobre os quais elas agem, mapeado para o ATT&CK [13]. Grafos construídos a partir de textos de inteligência de ameaças não capturam a estrutura do tráfego em tempo real, e ainda não está claro como aplicá-los em operações reais de ataque e de defesa [14].

Trabalhos mais recentes alimentam os grafos com o próprio tráfego, e entre eles o KLAGÉ [8] é o mais próximo da abordagem aqui proposta. Modelos aprendidos também avançam em tarefas de gestão afins. Uma linha de trabalho identifica dependências entre dispositivos por predição de ligações em grafos de rede [15], e outra aprende sequências de ações de ataque a partir de medições em bancada, com modelos estatísticos [16]. Na detecção de *bots* baseada em grafos, os *hosts* de um grafo de fluxos são classificados por atributos de grau, de agrupamento e de centralidade [17]. As três linhas operam sobre entidades de nível de rede, e nenhuma deriva um escopo de mitigação do que encontra. Da mesma forma, detectores no nível de contas encontram coordenação ao agrupar contas que agem em sincronia [18].

### B. Detecção de DDoS de Camada 7 sobre HTTP

O estado da arte em detecção de DDoS de camada 7 sobre HTTP [5], [19] se divide em duas categorias: (i) perfilamento estatístico e (ii) aprendizado de máquina. Na primeira, a Análise de Componentes Principais (PCA) é aplicada a estatísticas de fluxo [20] ou a uma matriz comportamental por sessão [9]. A abordagem por matriz de sessão é uma das primeiras formulações a pôr a sessão no centro, ainda que como vetor de atributos. Na segunda, mais recente, aplica-se aprendizado supervisionado a atributos de sessão [6], e os autores observam que o método não foi validado em tempo real e que sua saída é

binária, sem explicação ontológica. Nenhuma das duas categorias preserva relações entre sessões: a coordenação que houver se dissolve no vetor de atributos antes que o classificador possa vê-la. As defesas contra ataques que imitam picos legítimos de acesso (*flash crowds*) admitem os clientes que resolvem um desafio ou gastam banda, e medem o serviço que os usuários legítimos conservam [21], [22]. Derivar um filtro de um alarme é o objetivo da extração de anomalias, que isola os valores de atributos sobre-representados no tráfego anômalo [23], e da geração de assinaturas validadas num conjunto de tráfego normal [24]. Ambas operam sobre atributos de fluxo ou de *payload*, sem as impressões digitais dos clientes de uma aplicação.

### C. Modelagem de Sessão em Detecção de Anomalias

Nenhum dos 75 estudos de uma meta-análise sobre o tema [7] mantém a sessão como um objeto com identidade, alvo, comportamento e relações tipadas.

Já as plataformas industriais de gestão de *bots* exploram a estrutura entre sessões: sinais agregados por JA4 sobre o tráfego recente alimentam a pontuação de *bots* e a proteção contra DDoS [10]. Essas plataformas são proprietárias e não oferecem ontologia pública, *benchmark* acadêmico reproduzível nem métrica formal de dano colateral. O meio pelo qual um escopo trafegaria já é padronizado: o canal de sinalização do DOTS, do IETF, leva um pedido de mitigação do domínio atacado a um mitigador, com o escopo dado pelos recursos sob ataque. O DOTS tem ainda um canal de dados, que instala filtros sobre campos dos cabeçalhos de rede e de transporte, e uma telemetria, que reporta os maiores emissores por prefixo de origem [25]–[28]. Nenhum atributo do DOTS consegue expressar uma impressão digital de cliente da camada de aplicação, como o JA4.

A Tabela I deixa a fronteira explícita. Detectores estatísticos e de aprendizado de máquina reduzem a sessão a um vetor, e com isso nenhuma relação entre sessões sobrevive e nenhum escopo de mitigação ou custo pode decorrer da decisão. Detectores baseados em grafos preservam relações, mas entre entidades de rede e em forma aprendida, e plataformas industriais usam agregados por impressão digital sem divulgar a representação adotada nem o custo de agir com base neles. Nenhuma abordagem publicada reúne, portanto, três propriedades: a sessão de aplicação como unidade de raciocínio, relações entre sessões explícitas e tipadas, e um escopo de mitigação derivado da mesma representação que produziu o veredicto, com o dano colateral reportado. O restante do artigo apresenta essa combinação e mede quanto custa um escopo errado.

## III. GRAFO DE CONHECIMENTO CENTRADO NA SESSÃO

A Fig. 1 resume a abordagem. Quatro estágios operam sobre o tráfego ao vivo: os eventos HTTP são ingeridos, convertidos em instâncias da ontologia (com as sessões como nós persistentes do grafo), avaliados por regras e transformados num veredicto que carrega sua cadeia de evidência e seu escopo.

### A. O Uso de Grafos de Conhecimento

Um grafo de conhecimento registra fatos como triplas RDF (sujeito, predicado, objeto), e uma ontologia OWL 2 declara as classes e as propriedades que essas triplas podem usar. Trabalhamos no perfil OWL 2 RL [29], em que a inferência é feita por encadeamento de regras para frente (*forward chaining*) em tempo polinomial. Isso torna tratável, em tempo de execução, uma materialização limitada, o que o OWL 2 DL, mais expressivo, não permite. A materialização roda uma vez por janela, fora do processamento de cada requisição. O SWRL acrescenta regras de Horn, que deduzem uma tripla quando uma conjunção de premissas é verdadeira, e o SPARQL consulta e agrega o resultado. A identidade do cliente vem do JA4 [30], um *hash* da mensagem *ClientHello* do TLS que identifica a pilha TLS do cliente. Como não depende do endereço de origem, o JA4 resiste à rotação de IP.

### B. Definição da Ontologia

A classe central é *ApplicationSession*. Ela se relaciona por *hasIdentity* com uma *Identity* composta (*cookie*, *token* JWT, nome de usuário, impressão digital TLS), por *originatesFrom* com o *IPAddress* de origem, por *targets* com um *Endpoint* (*LoginEndpoint*, *APIEndpoint*, *StaticEndpoint*), por *exhibitsBehavior* com um perfil comportamental e por *mitigatedBy* com uma *Mitigation* (*RateLimitPolicy*, *ChallengeResponse*, *WAFRule*). A cadeia de evidência carrega o escopo dessa mitigação (Seção III-G). A classe abstrata *ApplicationLayerAttack* se especializa em *SlowHTTPDoSFamily*, que por sua vez se divide em *ConnectionExhaustionAttack* (*Slowloris*, *slow body*, *slow read*) e *CoordinatedHTTPFlood* (*HULK*, *GoldenEye*), a classe que nossa regra deriva. A ontologia está alinhada ao STIX 2.1 [31] e ao MITRE ATT&CK.

### C. Modelo de Ameaça

O defensor opera a aplicação atacada e observa, a cada requisição, o *ClientHello* TLS (e, portanto, uma impressão digital JA4), o endereço e o prefixo/ASN de origem, o *endpoint* alvo, o instante de chegada e todo material de identidade que a aplicação expõe. Ele não vê o canal de controle da *botnet* nem sabe quais origens são coordenadas. Pode, porém, traçar o perfil do tráfego do próprio serviço fora dos episódios de ataque. O atacante controla  $K$  dispositivos comprometidos e pode moldar cada sessão para que seja estatisticamente indistinguível de uma legítima, o que constitui a premissa de furtividade da nossa avaliação. O dano é o esgotamento da capacidade: cada sessão ocupa uma conexão e um *worker*, como faria uma sessão legítima. Nenhuma taxa por sessão sobe, e a taxa agregada cresce apenas na proporção da fatia que a campanha representa no tráfego do *endpoint*. Temporização, identidades e cargas úteis podem receber ruído, ser rotacionadas ou aleatorizadas, cada uma a um custo operacional que a ponderação da Seção III-D codifica. A pilha TLS é mais difícil de trocar, porque é fixada pela biblioteca embarcada em cada dispositivo, como nas *botnets* de roteadores, câmeras e DVRs da linhagem Mirai [32]. Para apresentar o *ClientHello* de um

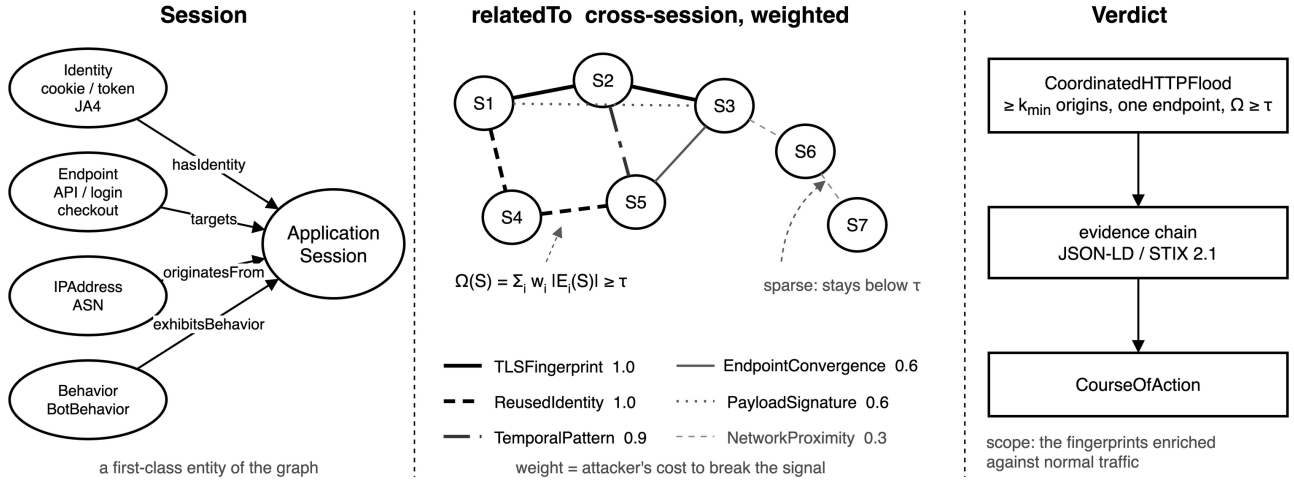


Fig. 1. Ontologia centrada na sessão e regra de detecção. A convergência de *endpoint* liga todo par de sessões do *endpoint*. Apenas alguns pares estão desenhados.

navegador, o atacante precisaria instalar em cada dispositivo uma biblioteca de imitação e mantê-la em dia com as versões dos navegadores. *Bots* que fazem isso, como fazendas de navegadores *headless* ou *bots* construídos sobre bibliotecas de imitação de TLS, ficam fora deste modelo, e as Seções V-C e VI os avaliam como a fronteira do modelo.

#### D. A Família *relatedTo* Ponderada

O vínculo entre sessões é modelado como uma família estruturada de relações. Cada membro é declarado com `rdfs:subPropertyOf` sob uma propriedade simétrica *relatedTo* e anotado com um peso *coordinationWeight*  $w_i \in [0,1]$  (Fig. 1). Cada sub-relação, isto é, cada sub-propriedade de *relatedTo*, é instanciada de forma independente, a partir da sua própria evidência observada, de modo que duas sessões podem estar ligadas por uma, por várias ou pelas seis. Três das seis entram nesta avaliação (Apêndice C).

Os pesos seguem o custo que o atacante tem para contornar cada sinal. A pilha TLS é fixada pela biblioteca de cada dispositivo, e o JA4 ordena as extensões antes de calcular o *hash*, de modo que continua estável sob a aleatorização de extensões que quebra o JA3 [30]. Já as *botnets* se espalham entre prefixos, e o NAT de operadora faz de um /24 compartilhado um mau discriminador, o que torna a proximidade de rede o elo mais fraco. O Apêndice B corrobora essa ordem e deixa em aberto os valores absolutos.

#### E. Regra de Detecção

Seja  $S_W$  o conjunto de sessões ativas na janela  $W$ . Para um subconjunto  $S \subseteq S_W$  e um *endpoint* candidato  $e$ , a massa de coordenação é

$$\Omega(S) = \sum_{i \in \mathcal{R}_{rel}} w_i |E_i(S)|, \quad (1)$$

em que  $E_i(S)$  é o conjunto de pares de origens distintas  $\{o(s_a), o(s_b)\}$ , com  $s_a, s_b \in S$ , cujas sessões estão ligadas pela sub-relação  $i$ , e  $o(s)$  é o endereço de origem de

$s$ . Cada par conta uma vez, para que as muitas sessões de um mesmo cliente não se passem por coordenação. A regra *CoordinatedHTTPFlood* vale quando existe  $S$  com (i) ao menos  $k_{min}$  origens; (ii)  $\langle s, targets, e \rangle$  para toda  $s \in S$ , isto é, todas as sessões visam o mesmo *endpoint*; e (iii)  $\Omega(S) \geq \tau_{cluster}$ . A regra só aciona a mitigação quando o teste de enriquecimento da Seção III-G aponta alguma impressão digital. Na avaliação por janela,  $S$  reúne todas as sessões que visam  $e$  (Apêndice E).

Na escala da janela, essa massa mede essencialmente volume. Sob a condição (ii), o termo de *endpoint* vale  $0,6 \binom{n}{2}$  para  $n$  origens e responde por 91% de  $\Omega$  na campanha da Listagem 2 (Apêndice D). Um limiar sobre o número de origens distintas da janela, calibrado da mesma forma que  $\tau_{cluster}$ , funciona, portanto, como gatilho pelo menos tão bem quanto  $\Omega$  (Apêndice E), e o discriminador vem do teste de enriquecimento.

#### F. Construção do Grafo em Tempo de Execução

Sessões e relações são mantidas numa janela operacional deslizante ( $W = 5$  min por padrão) e removidas de forma incremental, como no processamento de fluxos RDF, com materialização no Apache Jena Fuseki e armazenamento em TDB2 no protótipo. A avaliação calcula a mesma regra a partir das contagens de classe (Apêndices D e E). O Apêndice A traz o algoritmo de construção e o procedimento de cada sub-relação *relatedBy* para um par de sessões. Os casamentos exatos de JA4, de *endpoint* e de prefixo são relações de equivalência. Cada um divide  $S$  em classes, e  $|E_i(S)| = \sum_k \binom{n_k}{2}$ , em que  $n_k$  é o tamanho de cada classe, contado em origens. Admitir uma sessão se resume, então, a acrescentar sua origem a uma classe de cada sub-relação desse tipo, uma inserção em tabela *hash* cujo custo não depende de  $|S_W|$ . Assim, o cálculo de  $\Omega(S)$  dispensa arestas entre pares. Apenas as relações não transitivas (JA4 por variante próxima, temporal e de carga útil) comparam pares candidatos, e índices de prefixo e *hashing* sensível à localidade restringem esses pares. A Seção V-E mede os dois caminhos em função da ocupação da janela.

O raciocínio simbólico tem dois estágios (Listagem 1). O SWRL expressa cada sub-relação `relatedBy` como uma regra de Horn sobre os predicados que os extratores de evidência produzem, e os predicados numéricos, como a distância de *dynamic time warping* (DTW), entram como uma comparação embutida com um limiar. Como o SWRL não soma, uma consulta SPARQL agrega os tamanhos das classes com os pesos que lê das anotações `coordinationWeight`.

#### G. Cadeia de Evidência e Escopo de Mitigação Derivado

Quando a regra dispara, o motor emite a regra satisfeita, as instâncias envolvidas, a decomposição de  $\Omega(S)$  por sub-relação e o escopo de mitigação derivado. Exportamos esse conjunto em JSON-LD e em STIX 2.1, como um *indicator* e um *course-of-action* ligados por *mitigates*, para ingestão por ferramentas SIEM e SOAR.

A forma de escolher o escopo decide se o veredicto ajuda ou prejudica, e a escolha por frequência falha. Escolher a impressão digital modal do agrupamento, isto é, a mais frequente, é selecionar o que for comum, e num serviço sob ataque o que é comum é a população legítima. Quando a *botnet* se espalha por várias pilhas TLS, cada pilha do atacante é menor que a cabeça da distribuição benigna. O valor modal passa a ser uma impressão digital *legítima*, e o filtro resultante bloqueia usuários legítimos sem atingir nenhum atacante (Seção V-C).

Por isso selecionamos o discriminador por **enriquecimento**. Seja  $c(f)$  o número de origens com a impressão digital  $f$  no agrupamento que disparou a regra,  $n = \sum_f c(f)$ , e  $b(f)$  a prevalência de  $f$ , acrescida de  $1/N$ , entre os  $N$  pares origem-impressão digital de um perfil de tráfego normal que o operador mantém fora dos episódios de ataque. O escopo admite toda impressão digital  $f$  que seja, ao mesmo tempo, enriquecida,  $c(f)/n \geq \rho b(f)$ , e improvável sob a distribuição de referência,  $\Pr[X \geq c(f)] < \lambda_e/|F|$  para  $X \sim \text{Bin}(n, b(f))$ . O divisor de Bonferroni  $|F|$  conta as impressões digitais do perfil e do agrupamento. Em outras palavras,  $f$  entra no escopo quando a fração de origens do agrupamento que a apresentam é pelo menos  $\rho$  vezes a sua prevalência no tráfego normal e quando uma contagem tão alta seria improvável por acaso. O resultado é um *conjunto* de impressões digitais, capaz de cobrir uma *botnet* fragmentada. No tráfego de produção, frotas de clientes legítimos que entram em atividade ao mesmo tempo quebram a hipótese de independência da binomial (Apêndice E). Calibramos, então, o nível de significância  $\lambda_e$  por *endpoint*  $e$ , do mesmo modo que  $\tau_{\text{cluster}}$ : ele é o maior valor, até 0,01, com o qual o escopo produz um filtro em no máximo 1% das janelas livres de ataque. Isso impõe ao teste um piso de calibração. Entre  $n$  origens, ele só aponta uma pilha a partir da menor contagem  $c$  com  $\Pr[X \geq c] < \lambda_e/|F|$ , de modo que a menor *botnet* que ele consegue apontar cresce à medida que o nível cai (Seção V-D). Como as frotas também tornam as contagens sobredispersas, avaliamos ainda uma variante cuja distribuição de referência é uma beta-binomial ajustada para cada impressão digital nas janelas de calibração. Usamos  $\rho = 3$ . No tráfego gerado, em que cada sessão tem origem própria,  $\lambda_e$  permanece em 0,01.

O perfil de referência precisa vir de fora do episódio de ataque, porque a campanha ocupa toda a janela e, de outro modo, daria a todo candidato um enriquecimento igual a um. Nenhum rótulo é usado: o perfil é o que um defensor observa em operação normal. Um adversário que se esconde atrás de uma impressão digital benigna popular, portanto, não aparece enriquecido, e a abordagem se abstém de definir um escopo e informa que não há discriminador.

#### IV. METODOLOGIA DE AVALIAÇÃO

Cada fonte de tráfego responde a uma pergunta. Um gerador calibrado com sessões benignas do CICIDS2017 e com JA4 medidos numa CDN de produção fornece as campanhas controladas da avaliação principal. Capturas de laboratório testam ataques convencionais e permitem a comparação com o sistema publicado mais próximo, e oito dias de tráfego dessa CDN medem os falsos alarmes da regra e, com uma *botnet* injetada, sua capacidade de detecção. Em seguida apresentamos as linhas de base, as configurações da ablação e as métricas.

##### A. Cenários e Conjuntos de Dados

A avaliação é parametrizada pelo *grau de distribuição*  $K$  da campanha, isto é, o número de dispositivos que o atacante controla. Reportamos  $K = 50$ , uma *botnet* pequena cujas origens ficam, cada uma, dentro do próprio limite de taxa, e  $K = 1000$ , uma campanha espalhada por centenas de ASNs e prefixos, com sinal por sessão desprezível e a sub-relação `relatedByNetworkProximity` quase vazia. Uma origem única fica a cargo dos detectores volumétricos.

**Avaliação principal: tráfego sintético calibrado.** Um gerador produz campanhas distribuídas de Slow HTTP DoS, a família da Seção III-B: sessões sustentadas, de baixa taxa, contra um único *endpoint*, com a coordenação conhecida de antemão (verdade de referência). As campanhas são parametrizadas por  $K$ , pela fração de origens que compartilham impressão digital TLS e identidade, pela dispersão de IP e de ASN e pela janela de tempo.

Dois parâmetros do gerador decidem se o cenário se aproxima do tráfego de produção: a popularidade das impressões digitais benignas, que medimos, e a composição da *botnet*, que nenhuma medição publicada fixa e que por isso variamos. O primeiro segue uma curva de Zipf de expoente  $\alpha$ . Populações reais são concentradas, já que um navegador atualizado numa dada plataforma emite essencialmente um único JA4 [30]. Calibramos essa curva com 6,33 milhões de requisições TLS registradas em produção num ponto de presença de borda da Azion Technologies<sup>1</sup>: 495 impressões digitais distintas, com a mais comum respondendo por 38,4% das requisições e as dez primeiras por 93,8%. A curva medida fica entre  $\alpha = 1,5$  e  $\alpha = 2,0$ . Tomamos  $\alpha = 1,5$  como valor canônico e varremos a faixa. Quanto à composição, *botnets* reais abrangem classes

<sup>1</sup>A medição foi feita em 22 de agosto de 2026, a partir de um *log* de acesso que a plataforma já coleta. O extrato guarda apenas impressões digitais de software cliente e suas frequências, sem endereço, *host*, URI, cabeçalho ou localidade.

de dispositivo como roteadores, câmeras IP e DVRs [32] e, com elas, várias pilhas TLS. Distribuímos 90% da campanha de maneira uniforme por  $M$  pilhas, a escolha conservadora para um dado  $M$ , e o restante por impressões digitais únicas. Tomamos  $M = 25$  como valor canônico. Um modo *adversarial* deixa a *botnet* adotar as impressões digitais benignas mais comuns, como faria um *bot* que imita um navegador, e marca a fronteira do modelo de ameaça. O Apêndice B descreve a medição e suas ressalvas.

As distribuições legítimas são calibradas com as  $\sim 322$  mil sessões benignas do CICIDS2017 e verificadas por testes de Kolmogorov–Smirnov:  $D = 0,003$  para a duração e  $D = 0,002$  para a contagem de requisições, ambos com  $p > 0,8$ . Com isso, as distribuições marginais geradas coincidem com as do CICIDS2017. Um modo *furtivo* sorteia as sessões do atacante das mesmas distribuições do tráfego benigno, de modo que a campanha só se revela pela estrutura entre sessões.

**Capturas de laboratório.** O CICIDS2017 [33] (captura de quarta-feira: Slowloris, Slowhttptest, HULK, GoldenEye) e o CIC-IoT2023 [34] (DDoS Slowloris e HTTP Flood) passam pelo mesmo *pipeline* de extração (PCAP  $\rightarrow$  JA4 + fluxos  $\rightarrow$  sessões  $\rightarrow$  grafo). O CIC-IoT2023 é o conjunto em que o KLAGe reporta  $F_1 = 84,1\%$  para DDoS Slowloris. **Tráfego de produção.** Usamos oito dias de tráfego da CDN da Azion, na forma de contagens de origens distintas, de conexões e de pares no mesmo /24, por impressão digital e por janela de cinco minutos, e um nono dia, analisado com todas as escolhas fixadas de antemão (Seção V-D).

### B. Linhas de Base, Configurações e Métricas

Cada configuração da ablação roda com  $n = 30$  sementes distintas, com cada campanha dividida em 70/30 entre sessões de treino e de teste, o que fornece intervalos de confiança por *bootstrap* e permite testes pareados. Todas as configurações consomem os mesmos atributos de tráfego. Assim, com a disponibilidade de dados fixa, a ablação isola o efeito da *representação*. Em cada cenário,  $\tau_{\text{cluster}}$  é fixado no percentil 99 de  $\Omega$  sobre agrupamentos legítimos, e  $\lambda_e$  de modo que o escopo produza um filtro em no máximo 1% deles.

A linha de base por sessão é deliberadamente **forte**: uma *Random Forest* (RF) sobre todos os oito a nove atributos de fluxo disponíveis (contagem e duração das requisições, somas de bytes e de pacotes nos dois sentidos, média e desvio padrão do intervalo entre requisições). Para que o resultado negativo não dependa de uma única classe de modelos, repetimos o experimento com *gradient boosting* por histograma, um *perceptron* multicamadas e regressão logística sobre os mesmos atributos. Reimplementamos também três linhas de base acadêmicas: perfilamento por PCA [20], PCA seguida de agrupamento por  $k$ -médias sobre uma matriz comportamental [9] e RF/SVM supervisionados sobre atributos de sessão [6]. Também comparamos nossos resultados com os números publicados do KLAGe no mesmo conjunto de dados e no mesmo ataque. As quatro configurações da ablação são: **(a)** a linha de base de aprendizado de máquina, sem ontologia; **(b)** a ontologia *sem* sub-relações *relatedBy*,

com as sessões como nós isolados; **(c)** a ontologia *apenas* com *relatedByNetworkProximity*, que emula um detector ingênuo por ASN ou prefixo; e **(d)** a representação completa entre sessões, sobre as três sub-relações observáveis no tráfego avaliado. Assim, (d)–(a) mede a contribuição total, (d)–(b) o ganho de representar a estrutura entre sessões, e (d)–(c) o ganho dos sinais de peso alto sobre a proximidade de rede.

O escopo é comparado com escopos mais simples, sobre o mesmo alarme: a impressão digital modal do agrupamento; um filtro das impressões digitais ausentes do perfil e vistas em ao menos  $k_{\min}$  origens (*inéditas*); e um  $z$ -score por impressão digital em relação ao perfil ( $z > 3$ ). No tráfego de produção, esse  $z$ -score e um segundo, que compara cada impressão digital com o seu próprio histórico de calibração, também rodam em versão calibrada. Seus limiares são elevados, por *endpoint*, até o orçamento do teste de enriquecimento: um filtro em no máximo 1% das janelas livres de ataque. As pilhas que o gerador atribui à *botnet* nunca aparecem no vocabulário benigno. Um modo *compartilhado* as sorteia do perfil, excluindo as suas dez impressões digitais mais comuns, como faz a injeção no tráfego de produção.

A configuração (d) consome a evidência entre sessões na forma de *atributos*: a fração do agrupamento que compartilha o mesmo JA4 ou o mesmo /24, mais o seu tamanho. Esse substituto da família *relatedBy* em nível de atributos mantém todas as configurações iguais em dados e em classificador.

Além das métricas usuais de classificação, reportamos o **dano colateral**: a fração do tráfego legítimo que cai dentro do escopo derivado do veredicto. É a taxa de falsos positivos do filtro que um operador de fato aplicaria, e ela pode diferir muito da taxa do detector (Seção V-C).

## V. RESULTADOS

Os resultados seguem a ordem das afirmações que sustentam: a ablação sob campanhas furtivas distribuídas, a regra e seu escopo em tráfego gerado, o dano colateral do escopo, o escopo no tráfego de produção e o custo de execução.

### A. Ablação em Campanhas Furtivas Distribuídas

A detecção por sessão fica no nível do acaso em campanhas furtivas distribuídas nas quais os usuários legítimos acessam o *mesmo endpoint* e a mesma porta do ataque, de modo que nem o volume de fluxo nem a porta de destino separam as classes. Em  $K = 50$  e  $K = 1000$ , a configuração (a), com o conjunto completo de atributos de fluxo, alcança uma ROC AUC por sessão de 0,498 e 0,503, a configuração (b) fica em 0,500 e 0,502, e as três linhas de base acadêmicas variam de 0,495 a 0,518. A configuração (a) permanece entre 0,488 e 0,503 com qualquer família de classificador ( $\alpha = 1,5$ ,  $M = 25$ ,  $n = 30$  sementes). É o esperado, porque cada sessão do atacante é sorteada, por construção, das distribuições benignas, o que situa o colapso na representação. A representação entre sessões (d) alcança 0,927 e 0,982, e a configuração (c), restrita à proximidade de rede, apenas 0,499 e 0,659, porque a *botnet* está dispersa. Testes de Wilcoxon pareados, com correção de Bonferroni para quatro comparações, indicam

Tabela II  
O ESCOPO POR ENRIQUECIMENTO DO AGRUPAMENTO DE MAIOR  $\Omega$  COMO DETECTOR.

| Cenário              | Regra, sem rótulos |       |              | Aprendido, TFP = 0 |       |          |
|----------------------|--------------------|-------|--------------|--------------------|-------|----------|
|                      | Revoc.             | TFP   | $F_1$        | Inéditas           | (d)   | + perfil |
| Monolítica ( $M=1$ ) | 89,8%              | 0,00% | 0,946        | 89,8%              | 91,7% | 85,9%    |
| $M=5$                | 90,0%              | 0,00% | 0,948        | 90,0%              | 88,6% | 91,7%    |
| $M=25$               | 90,3%              | 0,00% | <b>0,949</b> | 90,3%              | 36,4% | 87,4%    |
| $M=100$              | 38,6%              | 0,00% | 0,556        | 88,1%              | 17,6% | 86,9%    |
| $M=25$ , compart.    | 85,4%              | 2,23% | 0,910        | 0,0%               | —     | —        |
| $M=25$ , adversarial | 30,4%              | 3,78% | 0,452        | 0,0%               | 7,8%  | 6,6%     |

TFP: taxa de falsos positivos. Nas colunas dos modelos aprendidos, a revocação com TFP = 0. Inéditas: revocação do filtro de impressões digitais ausentes do perfil (TFP de no máximo 0,03%). Compart.: pilhas compartilhadas, tiradas da cauda do perfil (tiradas, em vez disso, do vocabulário do gerador, em boa parte ausente do perfil amostrado, elas deixam o filtro de impressões inéditas em 84,4% e a regra em 89,8%). Os modelos aprendidos, que treinam com os rótulos da campanha (o segundo também lê o perfil da regra), não rodam nesse modo.  $K = 1000$ ,  $n = 15$  sementes.

que as diferenças **(d)–(c)** e **(d)–(a)** são significativas em  $K = 1000$  ( $p_{\text{Bonf}} = 7,5 \times 10^{-9}$ ,  $d$  de Cohen de 13,5 e 22,3). Como as sementes vêm de um único gerador, os testes mostram uma diferença estável entre os sorteios que ele produz.

Essa separabilidade não se generaliza para outro número de pilhas. Quando a *botnet* se fragmenta em  $M$  pilhas, a evidência entre sessões se torna *não monotônica* em relação ao rótulo: em  $K = 1000$ , um atacante compartilha a impressão digital com algumas dezenas de outras sessões, e um cliente legítimo da cabeça da distribuição benigna, com centenas. O atacante fica, assim, numa faixa intermediária. Comitês de árvores conseguem isolar essa faixa (0,92 a 0,99), o *perceptron* chega a 0,86 e 0,95, e a regressão logística, cuja resposta a cada atributo é monotônica, fica em 0,87 e 0,80. Treinada em campanhas de cinco pilhas e testada em campanhas de 25 ou de 100, a configuração (d) cai para 0,61 e 0,63. Treinada em 25 e testada em cinco, cai para 0,48 (Apêndice C): a *Random Forest* aprende a faixa que um dado  $M$  produz. A regra dispensa rótulos: ela lê um perfil de tráfego normal e mantém uma revocação de cerca de 90% para todo  $M$  até 25 (Tabela II).

O fato de (b) também ficar no nível do acaso mostra que atributos por sessão não bastam e que a detecção depende da evidência entre sessões. Os sete agrupamentos de ataque das duas capturas de laboratório (seis ataques e a classe DoS-Other do CICIDS2017) vêm de uma a sete origens num único /24. São inundações que um limite por prefixo conseguiria conter. Nessas capturas, basta um classificador forte por sessão ( $F_1 = 0,900$  no DDoS Slowloris do CIC-IoT2023, Apêndice C). Por isso, a cadeia de evidência da Listagem 2 é a de uma campanha canônica gerada.

### B. A Própria Regra como Detector

Todos os números até aqui vieram de um classificador e nada dizem sobre a camada simbólica. Por isso avaliamos a regra diretamente: as sessões que se enquadram no escopo da Seção III-G são o conjunto sinalizado, sem treino de modelo e sem rótulo, e a decisão chega acompanhada da sua derivação.

Como a AUC avalia uma ordenação e a regra toma uma decisão, forçamos o classificador a operar no mesmo ponto que a regra (Tabela II).

Contra uma *botnet* monolítica ou pouco fragmentada, o modelo aprendido, operando sem falsos positivos, é competitivo (91,7% e 88,6% de revocação, contra 89,8% e 90,0% da regra). Com 25 pilhas a regra abre vantagem, 90,3% contra 36,4%, e a mantém diante de um modelo que opera com 1% de taxa de falsos positivos (66,8%). Esse modelo ordena bem as sessões (AUC 0,979), mas seus escores se sobrepõem aos do tráfego benigno na cauda. Quando recebe o perfil da regra como dois atributos por sessão, a prevalência da impressão digital da sessão e seu enriquecimento no agrupamento, o modelo recupera 87,4% com 25 pilhas e 86,9% com cem. Com cem pilhas, cada pilha é pequena demais para que o teste a aponte diante de um perfil de mil sessões, e a regra fica em 38,6% (Seção V-C). O modelo, porém, treina com os rótulos da campanha que avalia, e nenhum operador dispõe desses rótulos durante um ataque. A regra o iguala até 25 pilhas sem rótulo algum. Na linha adversarial, a própria regra tem taxa de falsos positivos de 3,78%, e o modelo chega a 23,4% de revocação quando opera com 1% de falsos positivos. O filtro de impressões inéditas tem o mesmo desempenho da regra até 25 pilhas, porque as pilhas geradas nunca aparecem no vocabulário benigno, e desempenho superior com cem pilhas (88,1%). Em pilhas tiradas da cauda do perfil, que clientes reais também apresentam, esse filtro não bloqueia nenhum atacante. A regra bloqueia 85,4% do ataque com 2,23% de dano colateral, e um  $z$ -score não calibrado, 89,9% com 3,37%.

Na Tabela II, o agrupamento de maior  $\Omega$  é tratado como se tivesse disparado a regra, sem exigir  $\Omega(S) \geq \tau_{\text{cluster}}$ . Avaliada por janela de cinco minutos (Tabela V), a condição  $\Omega(S) \geq \tau_{\text{cluster}}$  sozinha dispara em 78 a 86% das janelas de ataque e em 0,6% das janelas limpas. Dispara também, porém, em 80 a 100% dos picos legítimos de acesso [35]. Exigir que o teste de enriquecimento aponte uma impressão digital elimina todos os falsos alarmes e mantém a detecção, sem dano colateral.

### C. Dano Colateral em Tráfego Legítimo

A mitigação só faz sentido como um par: o que o escopo bloqueia e quanto isso custa (Fig. 2,  $n = 15$  campanhas).

Para uma *botnet* monolítica, a regra por frequência e o enriquecimento concordam: 89,8% das sessões do atacante são bloqueadas sem dano colateral observado, enquanto um limite global de taxa no *endpoint* desconectaria todos os usuários legítimos. A regra por frequência, que usávamos antes, foi validada nesse regime, que não é representativo. A partir de cinco pilhas, a impressão digital modal do agrupamento passa a ser uma impressão digital *legítima*, e o efeito da regra se inverte: bloqueia 0,0% do ataque e atinge 39,0% do tráfego legítimo, uma perda líquida. Com uma população benigna mais concentrada ( $\alpha = 2,0$ ), esse dano colateral sobe para 61,1%.

O enriquecimento elimina essa falha. Com  $M = 5$  e  $M = 25$ , bloqueia 90,0% e 90,3% do ataque, **sem dano colateral observado** nas  $n = 15$  campanhas, e iguala o resultado da

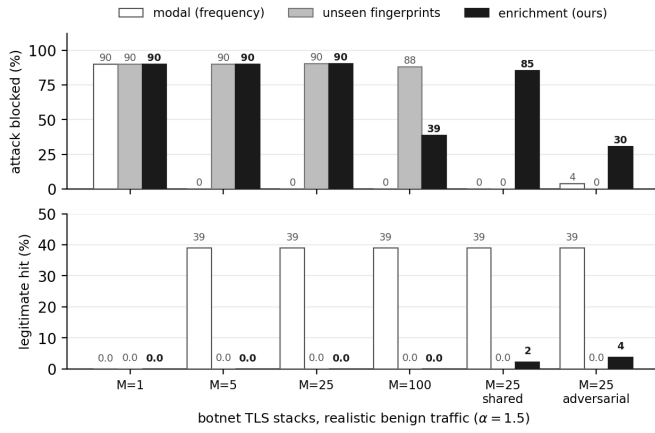


Fig. 2. Escopo: frequência, filtro de impressões inéditas e enriquecimento ( $\alpha = 1,5$ ,  $n = 15$ ).

regra por frequência no regime em que ela funciona. O filtro de impressões inéditas repete esse resultado nas pilhas geradas e o perde nas compartilhadas (Tabela II). Os 10% restantes são a cauda de atacantes com impressões digitais únicas: a mitigação delimitada por escopo troca completude por precisão. Com  $M = 100$ , a cobertura, isto é, a revocação do escopo, cai para 38,6%, ainda sem dano colateral. Uma pilha de cerca de dez sessões entre duas mil não se distingue da cauda de um perfil de 1000 sessões, e agregar ao perfil 30 períodos livres de ataque restaura 89,6%.

**Duas condições limitam o resultado.** A primeira é a fronteira do modelo de ameaça. Quando a *botnet* adota impressões digitais benignas comuns, poucas impressões digitais aparecem enriquecidas, e a regra bloqueia 30,4% do ataque com 3,78% de dano colateral, contra 3,6% e 39,0% da regra por frequência. Com um  $\rho$  mais estrito, o escopo se recusa a apontar uma impressão digital, o que é a resposta correta quando não existe discriminador. A segunda condição é o perfil de referência. Uma deriva moderada é tolerável (perfil com  $\alpha = 2,0$  contra um episódio com  $\alpha = 1,5$ : 90,3% de cobertura, sem dano colateral). Já um perfil achatado ou ausente faz toda impressão digital parecer rara, e o dano colateral salta para 77,6%. Um perfil atualizado e amplo é, portanto, requisito de implantação.

#### D. Tráfego de Produção

Oito dias de *logs* de acesso de quatro *endpoints* de serviços próprios de uma operadora de CDN testam o escopo sobre clientes reais (Tabelas III e IV). Cada um dos cinco últimos dias é avaliado com o perfil, os limiares e o nível dos dias anteriores, com a *botnet* canônica injetada nas contagens (Apêndice E). Quando a contagem é feita em conexões, o escopo produz um filtro em 60,1% das janelas limpas, porque um mesmo cliente abre muitas conexões. Em origens, produz um filtro em 26,9% delas no nível nominal, porque frotas de clientes legítimos entram em atividade ao mesmo tempo. Calibrar  $\lambda_e$  nos dias anteriores reduz a taxa da regra a 0,2%. Um limiar sobre as origens distintas, que dispara nas mesmas

Tabela III  
O ESCOPO NO TRÁFEGO DE PRODUÇÃO, POR *endpoint*.

| Escopo                       |           | Limpas     |      |       | Pilhas novas |      | Compart. |      |
|------------------------------|-----------|------------|------|-------|--------------|------|----------|------|
|                              |           | FA         | Col. | Flash | 0,1×         | 1×   | 0,1×     | 1×   |
| E1                           | binomial  | 0,0        | –    | 0,0   | 11,8         | 70,8 | 7,7      | 68,6 |
|                              | beta-bin. | 0,1        | 0,2  | 0,1   | 11,9         | 70,8 | 10,0     | 67,8 |
|                              | z-score   | 0,2        | 0,2  | 0,3   | 11,9         | 70,8 | 9,0      | 67,1 |
| E2                           | binomial  | 0,3        | 31,4 | 5,4   | 0,0          | 5,0  | 0,0      | 0,0  |
|                              | beta-bin. | 1,3        | 10,7 | 13,5  | 0,7          | 25,9 | 0,2      | 13,2 |
|                              | z-score   | 1,1        | 11,0 | 14,2  | 0,1          | 18,3 | 0,0      | 2,5  |
| E3                           | binomial  | 0,1        | 48,5 | 11,0  | 0,0          | 1,9  | 0,0      | 0,5  |
|                              | beta-bin. | 0,0        | –    | 6,6   | 0,3          | 22,7 | 0,3      | 13,9 |
|                              | z-score   | 0,1        | 48,5 | 2,0   | 0,3          | 20,3 | 0,0      | 4,8  |
| E4                           | binomial  | 0,0        | –    | 2,2   | 0,1          | 2,1  | 0,0      | 1,5  |
|                              | beta-bin. | 0,1        | 3,5  | 0,9   | 0,1          | 6,0  | 0,1      | 3,9  |
|                              | z-score   | 0,0        | –    | 0,3   | 0,1          | 6,0  | 0,0      | 1,2  |
| Todos                        |           | binomial   | 0,1  | 32,9  | 4,6          | 3,0  | 19,9     | 1,9  |
|                              |           | beta-bin.  | 0,4  | 10,5  | 5,3          | 3,3  | 31,4     | 2,7  |
|                              |           | z-score    | 0,4  | 10,8  | 4,2          | 3,1  | 28,8     | 2,3  |
| Dia novo, todos os endpoints |           | regra      | 0,2  | 41,2  | 1,0          | 0,0  | 17,3     | 0,0  |
|                              |           | binomial   | 0,2  | 41,2  | 1,1          | 0,0  | 20,4     | 0,0  |
|                              |           | beta-bin.* | 0,0  | –     | 4,3          | 0,0  | 32,5     | 0,0  |
|                              |           | z-score    | 0,2  | 41,2  | 0,4          | 0,0  | 31,4     | 0,0  |

Valores em %. Binomial: o teste de enriquecimento unido ao filtro de impressões inéditas, com as frotas conhecidas isentas; beta-bin.: o mesmo com uma referência beta-binomial; z-score: calibrado para o mesmo orçamento de falsos alarmes; todos sob o gatilho de origens distintas. Regra: a regra básica sob  $\Omega \geq \tau$ . FA: fração das janelas livres de ataque (5643 nos dias de teste, 1152 no dia novo) em que o escopo produz um filtro. Col.: mediana da fração dos clientes dessas janelas que o filtro bloqueia. Flash: FA com um pico legítimo de acesso (*flash crowd*) de 100 usuários. Nas demais colunas, a fração média bloqueada de uma *botnet* de 25 pilhas, do tamanho da janela mediana do *endpoint* (1×) ou de um décimo dela, em pilhas ausentes do tráfego (novas) ou tiradas do perfil, excluídas as suas dez impressões digitais mais comuns (compartilhadas). As impressões digitais únicas limitam essa fração a cerca de 90%. \*Construída depois da leitura do dia novo.

Tabela IV  
PISO DE CALIBRAÇÃO E CONCORDÂNCIA COM O WAF POR *endpoint*.

| Endpoint           | Origens  | $\lambda_e$ |           | Piso (× janela) |          |      | WAF  |
|--------------------|----------|-------------|-----------|-----------------|----------|------|------|
|                    |          | bin.        | beta      | bin.            | + frotas | beta |      |
| E1, beacons de RUM | $> 10^3$ | $10^{-60}$  | $10^{-4}$ | 0,16            | 0,07     | 0,03 | –    |
| E2, console web    | $< 10^2$ | $10^{-73}$  | $10^{-2}$ | 11,6            | 11,6     | 0,90 | 1,5% |
| E3, API            | $< 10^2$ | $10^{-14}$  | $10^{-2}$ | 4,1             | 3,5      | 1,4  | 84%  |
| E4, SSO            | $< 10^2$ | $10^{-4}$   | $10^{-2}$ | 4,2             | 4,2      | 2,8  | 96%  |

Origens por janela.  $\lambda_e$ : o nível calibrado sob a binomial e sob a beta-binomial. Piso: tamanho da menor *botnet* cujas 25 pilhas novas o teste, nesse nível, consegue apontar, como múltiplo da janela mediana (mediana sobre os cinco dias de teste). WAF: fração dos clientes que o escopo bloquearia e que o WAF também bloqueou (E1 não tem atividade de WAF).

janelas que  $\Omega$  em 86 a 99,6% dos casos, reduz os falsos alarmes à metade quando usado como gatilho, sem perda de detecção (6 contra 14 janelas).

**As frotas impõem um piso de calibração.** Elas levam  $\lambda_e$  a  $10^{-60}$  no *endpoint* mais movimentado, onde o teste binomial só aponta uma pilha a partir de 15 a 18 origens. A menor *botnet* de 25 pilhas que ele consegue apontar tem, assim, 16% da janela típica. Esse piso cai para 7% quando as cinco ou seis frotas conhecidas do *endpoint* (impressões digitais que o teste aponta em ao menos 5% das janelas de calibração) ficam isentas do escopo e da calibração do nível, o que eleva  $\lambda_e$  para cerca de  $10^{-21}$ . A isenção funciona como uma lista de exceções: um *bot* que apresente a impressão digital de uma

frota não entra no escopo. Nos três *endpoints* pequenos, o piso é de 4 a 12 vezes a janela inteira (Tabela IV).

**Uma referência que modela as frotas rebaixa o piso.** As frotas tornam sobredispersas as contagens por impressão digital. Tendo como referência uma beta-binomial cuja correlação dentro da janela é ajustada para cada impressão digital nas janelas de calibração (Apêndice E), o nível do *endpoint* mais movimentado sobe para  $7 \times 10^{-5}$ , nível em que três origens bastam para apontar uma pilha. Nos demais, o nível chega ao teto de 0,01. O piso cai para 3% da janela do *endpoint* mais movimentado e para 0,9 a 2,8 janelas nos demais *endpoints*. Nenhuma impressão digital se qualifica, então, como frota conhecida, e a lista de exceções se torna desnecessária. No console web, com essa referência, os falsos alarmes dos dias de teste passam do orçamento (1,3%), como passam os do *z-score* calibrado (1,1%).

**Acima do piso, um *z-score* calibrado iguala o teste.** No *endpoint* mais movimentado, todo escopo calibrado bloqueia de 67 a 71% de uma *botnet* do tamanho da janela típica, e de 8 a 12% de uma com um décimo desse tamanho, para a qual o gatilho raramente dispara. Nos *endpoints* pequenos, uma *botnet* do tamanho da janela fica abaixo do piso da binomial, que bloqueia de 2 a 5% dela em pilhas novas, enquanto a beta-binomial e um *z-score* calibrado para o mesmo orçamento bloqueiam de 6 a 26%. No agregado dos quatro *endpoints*, a configuração binomial dispara em 0,1% das janelas limpas (5 de 5 643). Ela bloqueia 38,4% e 77,7% de 100 e de 1 000 atacantes em pilhas novas, e 21,7% e 63,3% em pilhas compartilhadas, nas quais o filtro de impressões inéditas sozinho não bloqueia nenhum. O *z-score* calibrado e a beta-binomial disparam em 0,4% das janelas limpas (20 e 22 janelas) e bloqueiam tanto ou mais com 100 atacantes (54,7% e 59,8% em pilhas novas, 23,7% e 40,5% em compartilhadas). Seus falsos alarmes atingem uma fração menor dos clientes da janela (10,8% e 10,5%, contra 32,9%). A binomial tem uma taxa menor de falsos alarmes, ao custo de um piso mais alto. Nas janelas de ataque, todo escopo calibrado bloqueia, em mediana, 0% dos clientes legítimos.

**No regime furtivo, o gatilho deixa a *botnet* passar.** Uma *botnet* de um décimo da janela do *endpoint* mais movimentado fica acima do piso quando as frotas conhecidas ficam isentas, e o escopo a aponta em todas as janelas, em pilhas novas e compartilhadas. O gatilho de origens distintas, porém, dispara em 0,7 a 42% dessas janelas, conforme o dia, de modo que 11,8% dos atacantes em pilhas novas são bloqueados. Um gatilho acionado apenas pelo escopo, examinado *a posteriori*, bloqueia 89,6% deles em pilhas novas e 61,1% em pilhas compartilhadas, disparando em 1,0% das janelas limpas do *endpoint*. Com a beta-binomial, bloqueia 90,0% e 78,3%, disparando em 2,1% delas.

**Um dia novo é compatível com a taxa de falsos alarmes.** Um nono dia, exportado depois de todas as escolhas acima e analisado com a configuração e as métricas fixadas de antemão (Apêndice F), registra, para a configuração binomial, 2 falsos alarmes em 1 152 janelas limpas, o que é compatível com os dias de teste ( $P = 0,27$ ). Os dois ocorrem no console

web e bloqueiam, em mediana, 41,2% dos clientes da janela. Essa configuração bloqueia 38,9% e 73,4% de 100 e de 1 000 atacantes em pilhas novas, e 21,5% e 60,9% em pilhas compartilhadas. Com uma *botnet* de um décimo da janela do *endpoint* mais movimentado, o gatilho não disparou nenhuma vez. O *z-score* calibrado e a regra básica, termo de comparação do protocolo, dispararam nas mesmas duas janelas, e o *z-score* bloqueou mais com 100 atacantes (58,3% e 26,7%). Nos dias de teste, 4 dos 5 falsos alarmes do teste binomial coincidiram com os do *z-score* calibrado. A beta-binomial, construída depois da leitura do dia, não gerou nenhum. A consulta de contagens compilada devolveu, para o dia, números de origens distintas e de pares no mesmo /24 iguais aos da exportação em todas as 1 152 janelas.

#### E. Custo e a Escolha de $W$

Mantidas como classes (Apêndice D), as sub-relações baseadas em igualdade tornam o custo de admissão constante, cerca de  $0,37 \mu s$  por sessão, e a camada simbólica linear: 26,4 s para 100 000 sessões, contra 52,8 s para 1 000 com arestas entre pares. O valor de  $\Omega(S)$  ainda cresce com o quadrado do tamanho do agrupamento, mas o custo de calculá-lo deixou de crescer assim. A AUC de (d) é estável numa faixa de  $30 \times$  em  $W$ , porque a campanha gerada forma um único agrupamento em qualquer  $W$  (Apêndice D). A regra rodou com  $W = 300$  s.

### VI. DISCUSSÃO E LIMITAÇÕES

Discutimos o que limita os resultados e o que o grafo acrescenta (as limitações continuam no Apêndice C).

**O ganho depende do regime.** Limiares por IP resolvem inundações de origem única, e um classificador forte por sessão alcança  $AUC \geq 0,98$  em ataques de conjuntos públicos. Só campanhas distribuídas e furtivas precisam de evidência entre sessões, e só *botnets* acima do piso de calibração, que a distribuição de referência determina, podem ter o escopo definido por impressão digital. Abaixo do piso, resta recorrer a um limite de taxa ou a um desafio. Se uma campanha no nível do piso esgota o serviço depende da sua folga de capacidade, que não medimos.

**Cada avaliação sustenta uma afirmação distinta.** O gerador mostra o mecanismo sob uma imitação imposta por construção: as sessões do atacante tiram seus atributos de distribuições benignas reais, e por isso a inseparabilidade da Seção V-A não é artefato de uma linha de base enfraquecida. As pilhas do gerador nunca aparecem no vocabulário benigno, e é por isso que um filtro de impressões inéditas iguala o escopo nesse cenário. Nele, um modelo treinado com um número de pilhas falha quando testado com outro. O tráfego de produção mede os falsos alarmes sobre clientes reais e a detecção apenas para uma *botnet* injetada, ao longo de nove dias de uma única operadora. Nenhuma avaliação cobre uma campanha furtiva capturada.

**O escopo depende de um discriminador observável de peso alto.** À medida que cai a fração de origens que compartilham a impressão digital TLS, a  $AUC(d)$  decresce monotonicamente de 1,00 para  $\approx 0,74$  sob aleatorização

( $n = 10$ ). O escopo, que se apoia no JA4, perde então seu discriminador, enquanto o gatilho de volume continua funcionando. Ruído temporal e identidades rotacionadas não afetam o escopo. Um *bot* fora do modelo de ameaça que apresente o JA4 de um navegador anula o escopo. O Encrypted Client Hello [36] oculta o JA4 de quem observa no caminho, mas não de um defensor que termina o TLS, como supomos aqui.

**O que o grafo acrescenta.** O ganho de AUC da Seção V-A vem de representar a estrutura entre sessões, como a configuração (d) faz em atributos: um modelo linear sobre esses atributos ainda alcança 0,799, contra 0,495. O OWL não acrescenta nada a esse número nem ao gatilho, e o teste do escopo é estatístico. O grafo é a especificação que o operador implanta: a ontologia registra o que cada sub-relação de igualdade iguala, o seu peso e a unidade em que as classes são contadas. A consulta compilada a partir dela reproduz as contagens exportadas de origens e de pares no mesmo /24 em todas as janelas de dois dias de produção, e os tamanhos das classes de JA4 nas janelas em que o WAF não bloqueou nenhum cliente. Um novo sinal de igualdade chega à consulta como quatro triplas, sem mudança em nenhum código. O escopo e sua cadeia de evidência (Listagem 2) são exportados como pacotes STIX 2.1 que passam no validador da OASIS em modo estrito. Eles atravessam inalterados o servidor TAXII 2.1 de referência, exceto a definição de extensão, que esse servidor não consegue disponibilizar. O importador do MISP descarta as impressões digitais, que o STIX 2.1 só transporta numa extensão, e os filtros do DOTS só atuam sobre campos de rede e de transporte. Os dois alargariam o escopo para o *endpoint* inteiro ou para prefixos de endereço: a mitigação delimitada por impressão digital precisa de uma propriedade padrão para o JA4. Ainda não testamos se a derivação, que torna o veredicto inspecionável, reduz o tempo que um analista leva para decidir.

## VII. CONCLUSÃO E TRABALHOS FUTUROS

Tratamos o escopo de uma mitigação de DDoS na camada de aplicação como um teste calibrado sobre um grafo de conhecimento centrado na sessão. Em campanhas furtivas geradas, a detecção por sessão fica no nível do acaso, e atributos entre sessões alcançam ROC AUC de 0,93 a 0,98 apenas dentro de uma mesma estrutura de *botnet*. Definir o escopo pela propriedade mais comum do agrupamento seleciona uma impressão digital legítima assim que a *botnet* se espalha por várias pilhas TLS. Um teste binomial de enriquecimento, tendo o tráfego normal como referência, bloqueia 90% da *botnet* com até 25 pilhas, sem dano colateral observado e sem rótulo, e inclui no escopo as pilhas que clientes reais compartilham, o que um filtro de impressões inéditas não consegue. No tráfego de uma CDN, frotas legítimas impõem um piso ao que o teste calibrado consegue apontar: 16% da janela do *endpoint* mais movimentado sob a binomial e 3% sob uma beta-binomial que modela essas frotas, no ponto de operação de um  $z$ -score calibrado. Um dia novo é compatível com a taxa de falsos alarmes da configuração binomial, e, no regime furtivo,

o escopo aponta a *botnet* que o gatilho de volume, em sua maior parte, deixa passar. Os próximos passos são gatilhos acionados pelo escopo, a extensão a ataques sobre HTTP/2 e a avaliação com uma campanha furtiva capturada.

## AGRADECIMENTO

Agradecemos à Azion Technologies (<https://www.azion.com/>) por autorizar a medição de impressões digitais da Seção IV e a exportação das estatísticas agregadas de tráfego do Apêndice E, feita sem endereços, portas ou URIs.

## REFERÊNCIAS

- [1] National Institute of Standards and Technology, “CVE-2023-44487 detail,” National Vulnerability Database, 2023, accessed Sep. 25, 2026. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2023-44487>
- [2] J. Snellman and D. Iamartino, “How it works: The novel HTTP/2 ‘Rapid Reset’ DDoS attack,” Google Cloud Blog, Oct. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack>
- [3] L. Pardue, “HTTP/2 rapid reset: Deconstructing the record-breaking attack,” Cloudflare Blog, Oct. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/technical-breakdown-http2-rapid-reset-ddos-attack/>
- [4] O. Yoachimik, “Hyper-volumetric DDoS attacks skyrocket: Cloudflare’s 2025 Q2 DDoS threat report,” Cloudflare Blog, Jul. 2025, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/ddos-threat-report-for-2025-q2/>
- [5] E. Cambiaso, G. Papaleo, G. Chiola, and M. Aiello, “Slow DoS attacks: Definition and categorisation,” *International Journal of Trust Management in Computing and Communications*, vol. 1, no. 3–4, pp. 300–319, 2013.
- [6] C. Kemp, C. Calvert, T. M. Khoshgoftaar, and J. L. Leevy, “An approach to application-layer DoS detection,” *Journal of Big Data*, vol. 10, no. 1, p. 22, 2023.
- [7] M. Odusami, S. Misra, O. Abayomi-Alli, A. Abayomi-Alli, and L. Fernández-Sanz, “A survey and meta-analysis of application-layer distributed denial-of-service attack,” *International Journal of Communication Systems*, vol. 33, no. 18, p. e4603, 2020.
- [8] L. Belcastro, C. Carlucci, C. Cosentino, P. Liò, and F. Marozzo, “Enhancing network security using knowledge graphs and large language models for explainable threat detection,” *Future Generation Computer Systems*, vol. 176, p. 108160, 2026.
- [9] R. Bharathi and R. Sukanesh, “A PCA based framework for detection of application layer DDoS attacks,” *WSEAS Transactions on Information Science and Applications*, vol. 9, no. 12, pp. 389–398, 2012.
- [10] A. Martinetti, “Advancing threat intelligence: JA4 fingerprints and inter-request signals,” Cloudflare Blog, Aug. 2024, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/ja4-signals/>
- [11] Y. Jia, Y. Qi, H. Shang, R. Jiang, and A. Li, “A practical approach to constructing a knowledge graph for cybersecurity,” *Engineering*, vol. 4, no. 1, pp. 53–60, 2018.
- [12] V. Bonagiri, P. Priyadarshini, P. PanneerSelvi, S. Sahitya, R. Swamy, and P. Varalakshmi, “Practical applications of cybersecurity: A research paper on knowledge graphs for real-world security solutions,” in *Proc. 4th Asian Conf. on Innovation in Technology (ASIANCON)*, 2024.
- [13] P. E. Kaloroumakis and M. J. Smith, “Toward a knowledge graph of cybersecurity countermeasures,” The MITRE Corporation, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://d3fend.mitre.org/resources/D3FEND.pdf>
- [14] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, and Y. Zhou, “Recent progress of using knowledge graph for cybersecurity,” *Electronics*, vol. 11, no. 15, p. 2287, 2022.
- [15] L. Sadlek, M. Husák, and P. Čeleda, “Identification of device dependencies using link prediction,” in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2024, pp. 1–10.
- [16] X. Wang and R. Stadler, “IT intrusion detection using statistical learning and tested measurements,” in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2024, pp. 1–7.

- [17] A. Abou Daya, M. A. Salahuddin, N. Limam, and R. Boutaba, “A graph-based machine learning approach for bot detection,” in *Proc. IFIP/IEEE Symp. on Integrated Network and Service Management (IM)*, 2019, pp. 144–152.
- [18] Q. Cao, X. Yang, J. Yu, and C. Palow, “Uncovering large groups of active malicious accounts in online social networks,” in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 477–488.
- [19] N. Tripathi and N. Hubballi, “Application layer denial-of-service attacks and defense mechanisms: A survey,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–33, 2021.
- [20] G. Fernandes Jr., J. J. P. C. Rodrigues, and M. L. Proença Jr., “Autonomous profile-based anomaly detection system using principal component analysis and flow analysis,” *Applied Soft Computing*, vol. 34, pp. 513–525, 2015.
- [21] S. Kandula, D. Katabi, M. Jacob, and A. Berger, “Botz-4-Sale: Surviving organized DDoS attacks that mimic flash crowds,” in *Proc. USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2005.
- [22] M. Walfish, M. Vutukuru, H. Balakrishnan, D. Karger, and S. Shenker, “DDoS defense by offense,” in *Proc. ACM SIGCOMM*, 2006, pp. 303–314.
- [23] F. Silveira and C. Diot, “URCA: Pulling out anomalies by their root causes,” in *Proc. IEEE INFOCOM*, 2010.
- [24] Z. Li, M. Sanghi, Y. Chen, M.-Y. Kao, and B. Chavez, “Hamsa: Fast signature generation for zero-day polymorphic worms with provable attack resilience,” in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2006.
- [25] A. Mortensen, T. Reddy.K, F. Andreasen, N. Teague, and R. Compton, “DDoS Open Threat Signaling (DOTS) Architecture,” Internet Engineering Task Force, RFC 8811, Aug. 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8811>
- [26] M. Boucadair, J. Shallow, and T. Reddy.K, “Distributed Denial-of-Service Open Threat Signaling (DOTS) Signal Channel Specification,” Internet Engineering Task Force, RFC 9132, Sep. 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9132>
- [27] M. Boucadair and T. Reddy.K, “Distributed Denial-of-Service Open Threat Signaling (DOTS) Data Channel Specification,” Internet Engineering Task Force, RFC 8783, May 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8783>
- [28] M. Boucadair, T. Reddy.K, E. Doron, M. Chen, and J. Shallow, “Distributed Denial-of-Service Open Threat Signaling (DOTS) Telemetry,” Internet Engineering Task Force, RFC 9244, Sep. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9244>
- [29] W3C OWL Working Group, “OWL 2 web ontology language profiles, 2nd ed.” W3C Recommendation, 2012, accessed Sep. 25, 2026. [Online]. Available: <https://www.w3.org/TR/owl2-profiles/>
- [30] J. Althouse, “JA4+ network fingerprinting,” FoxIO Blog, Sep. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://blog.foxio.io/ja4+-network-fingerprinting>
- [31] OASIS, “STIX version 2.1,” OASIS Standard, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>
- [32] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, 2017, pp. 1093–1110.
- [33] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [34] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [35] J. Jung, B. Krishnamurthy, and M. Rabinovich, “Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites,” in *Proc. 11th Int. World Wide Web Conf. (WWW)*, 2002, pp. 293–304.
- [36] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, “TLS Encrypted Client Hello,” Internet Engineering Task Force, RFC 9849, Mar. 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9849>
- [37] M. Shafi, A. H. Lashkari, V. Rodriguez, and R. Nevo, “Toward generating a new cloud-based distributed denial of service (DDoS) dataset and

cloud intrusion traffic characterization,” *Information*, vol. 15, no. 4, p. 195, 2024.

## APÊNDICE A

### CONSTRUÇÃO EM TEMPO DE EXECUÇÃO E INSTANCIAÇÃO DAS SUB-RELAÇÕES

**Algoritmo 1** Construção do grafo de conhecimento em tempo de execução

**Requer:** fluxo de requisições  $\mathcal{R}$ , ontologia  $\mathcal{O}$ , janela  $W$   
**Garante:** grafo de conhecimento  $KG$  persistente ao longo de  $W$

```

1:  $KG \leftarrow \emptyset$ 
2: para cada requisição  $r \in \mathcal{R}$  faça
3:    $s, i, e \leftarrow \text{RESOLVESSÃO/IDENT./ENDPOINT}(r)$ 
4:    $KG.\text{add}(s, i, e); KG.\text{link}(s, \text{hasIdentity}, i)$ 
5:    $KG.\text{link}(s, \text{targets}, e)$ 
6:   para cada  $s' \in S_W$  ativa,  $s' \neq s$  faça
7:     para cada sub-relação  $i \in \mathcal{R}_{\text{rel}}$  faça
8:       se  $\text{VALE}_i(s, s')$  então
9:          $KG.\text{link}(s, \text{relatedBy}_i, s')$ 
10:      fim se
11:    fim para
12:  fim para
13:   $KG.\text{purgeOutside}(W)$ 
14: fim para
15: retorne  $KG$ 

```

O Algoritmo 1 constrói o grafo sobre a janela deslizante. Para maior clareza, ele enumera pares. Na prática, as relações baseadas em igualdade são mantidas como contagens de classe (Seção III-F), e, para as demais, os procedimentos a seguir decidem, para cada par candidato  $(s_a, s_b)$ , se a aresta é instanciada.

**TLSFingerprint.** A sub-relação é instanciada em dois estágios: por igualdade exata do JA4 completo ou, na falta dela, por variantes próximas (mesmo prefixo de transporte, versão TLS e ALPN, com no máximo um símbolo de diferença nos blocos ordenados de cifras ou de extensões), anotadas com  $ja4\_distance = 1$ . Isso absorve mudanças de versão dentro de uma mesma biblioteca cliente sem confundir pilhas distintas. O custo é  $O(1)$  por par.

**ReusedIdentity.** Esta sub-relação e as duas seguintes estão especificadas, mas não entram na avaliação. Ela é instanciada quando há qualquer *cookie*, *token* ou nome de usuário compartilhado. JWTs são comparados pela *claim* sub, e *cookies* e nomes de usuário são normalizados. O custo é  $O(1)$  amortizado, com índice invertido.

**TemporalPattern.** Seja  $\Delta(s)$  a sequência de intervalos entre chegadas de uma sessão com ao menos três requisições. A aresta é instanciada quando  $\text{DTW}(\Delta(s_a), \Delta(s_b)) / \max(|\Delta(s_a)|, |\Delta(s_b)|) \leq \tau_{\text{DTW}}$ . O DTW mede a distância entre duas sequências depois de alinhá-las de forma elástica, esticando ou comprimindo trechos de uma em relação à outra, e por isso atribui distância pequena a sessões de mesmo ritmo, ainda que uma delas tenha uma

```

# SWRL (Horn): instancia uma sub-relação
ApplicationSession(?a) ^ ApplicationSession(?b) ^
differentFrom(?a,?b) ^
tlsJa4(?a,?j) ^ tlsJa4(?b,?j)
-> relatedByTLSFingerprint(?a,?b)

# SPARQL: agregação ponderada a partir do tamanho
# das classes, contado em origens distintas
SELECT ?e (SUM(?w * ?n * (?n - 1) / 2) AS ?omega)
WHERE {
  { SELECT ?e ?c (COUNT(DISTINCT ?o) AS ?n)
    WHERE { ?s kg:targets ?e ; kg:inClass ?c ;
            kg:originatesFrom ?o . }
    GROUP BY ?e ?c }
  ?c kg:ofRelation ?rel .
  ?rel rdfs:subPropertyOf kg:relatedTo ;
      kg:coordinationWeight ?w .
  ?e kg:tauCluster ?tau . }
GROUP BY ?e ?tau
HAVING (SUM(?w * ?n * (?n - 1) / 2) >= ?tau)

```

Listagem 1. Veredicto como derivação: uma regra de Horn e a agregação SPARQL.

pausa ou uma requisição a mais. Como  $\Delta(s)$  é formado por intervalos, o instante em que cada sessão começa não interfere. O FastDTW com uma banda de Sakoe–Chiba torna o custo linear, e o *hashing* de vetores-resumo evita comparar todos os pares.

**PayloadSignature.** A aresta é instanciada quando a similaridade de cosseno entre os vetores formados pela média e pelo desvio padrão do tamanho do corpo e pelos *hashes* do User-Agent e do Content-Type dominantes excede  $\tau_{\text{payload}}$ .

**EndpointConvergence.** A sub-relação é instanciada quando as duas sessões convergem para o mesmo *endpoint*, com os caminhos normalizados em modelos (`/api/users/12345`  $\rightarrow$  `/api/users/{id}`). Quando o caminho coincide exatamente, a aresta recebe a anotação *exact = true*. O custo é  $O(1)$ , com índice em árvore de prefixos.

**NetworkProximity.** A sub-relação é instanciada quando há prefixo IPv4 /24 (ou IPv6 /48) compartilhado, ASN compartilhado, segundo tabelas BGP atualizadas, ou ambos. Quando ambos valem, a aresta é anotada com *strong = true*. O custo é  $O(1)$ , com índice invertido de prefixos.

## APÊNDICE B CALIBRAÇÃO DOS PESOS

A calibração depende do realismo do tráfego. Em capturas de laboratório, que têm algumas dezenas de impressões digitais JA4 distintas, a otimização chega a inverter o sinal do indicador TLS, porque as sessões benignas compartilham impressões digitais mais do que as campanhas parcialmente coordenadas.

No cenário realista, os usuários acessam o serviço atacado com impressões digitais tiradas de um conjunto de 2 000, cuja forma de Zipf é fixada pelas 495 impressões digitais medidas num ponto de presença. Esse conjunto excede as 76 a 818 impressões digitais que cada *endpoint* de produção mostra, de modo que a cauda benigna gerada é mais longa que a observada. Calibramos os pesos maximizando a ROC AUC por sessão entre ataque e tráfego benigno sobre o escore

$w_{\text{tls}} z(\text{ja4}) + w_{\text{ep}} z(\text{conv. endpoint}) + w_{\text{net}} z(\text{prox. rede})$ , com atributos padronizados, na grade  $\{0,3; \dots; 1,0\}^3$  (125 combinações, 60 cenários, 91 500 sessões). O melhor vetor é  $(w_{\text{tls}}=1,0; w_{\text{ep}}=0,3; w_{\text{net}}=0,3)$ , e a impressão digital TLS é o único sinal discriminativo por si só (AUC isolada de 0,93, contra 0,50 da convergência de *endpoint* e 0,58 da proximidade de rede). Os pesos da Fig. 1, (1,0; 0,6; 0,3), atingem a mesma AUC ótima de 0,943 e são insensíveis a perturbações de  $\pm 20\%$ . A calibração corrobora, portanto, a *ordem* dos pesos. Os pesos médio e baixo não são identificáveis separadamente, e a regra por janela detecta igualmente bem com pesos uniformes (Apêndice E), de modo que seus valores absolutos seguem em aberto. A medição de popularidade é ponderada por requisição, o que, se introduz algum viés, é o de *superestimar* a concentração. A cabeça da distribuição medida se ajusta a  $\alpha = 1,5$ , e as dez primeiras impressões digitais, a  $\alpha = 2,0$ .

## APÊNDICE C LIMITAÇÕES ADICIONAIS

**Instrumentação de sessão e identidade.** A abordagem supõe que sessões, identidades e impressões digitais TLS possam ser extraídas dos eventos de tráfego, e uma instrumentação parcial ou ruidosa a degrada. Contar origens pelo endereço de origem subestima o número de usuários atrás de um mesmo NAT de operadora e superestima o de clientes que rotacionam endereços de privacidade IPv6, cuja unidade natural é o /64.

**Custo de manutenção do grafo.** Os números da camada simbólica do Apêndice D são um limite superior, e a execução sobre o tráfego de produção do Apêndice E é *offline*, sobre contagens agregadas por *endpoint*. Uma implantação em operação numa CDN ainda exige o particionamento do Apêndice D, que não implementamos.

**Cobertura.** Ataques sobre HTTP/2 e sobre outros protocolos de aplicação (DNS, gRPC) exigem sub-relações além das da família Slow HTTP DoS sobre HTTP/1.1, e  $\Omega(S)$  instancia apenas as três sub-relações que o gerador e as capturas expõem. Reaproveitamento de identidade, padrão temporal e assinatura de carga útil estão especificados (Apêndice A), mas não entram na avaliação, de modo que seus pesos não foram testados.

**Amplitude dos conjuntos de dados.** A validação em laboratório cobre um conjunto e um ataque na comparação com o KLAGe, mais seis ataques do CICIDS2017 e do CIC-IoT2023. Não encontramos nenhuma captura independente do regime furtivo na camada 7: o BCCC-cPacket-Cloud-DDoS-2024 aponta a fraca representação da camada de aplicação como deficiência dos conjuntos que analisa [37], mas seus 17 cenários de DDoS são todos baseados em TCP. O RT-IoT2022, o CIC-DDoS2019 e uma campanha furtiva capturada ficam para trabalho futuro.

**Capturas de laboratório e KLAGe.** No DDoS Slowloris do CIC-IoT2023, cujos agrupamentos de ataque concentram sete origens num único dos 143 prefixos /24, uma linha de base por sessão *enxuta* (três atributos) desaba para  $F_1 = 0,179$  (AUC de 0,551). Uma linha de base *forte* (oito atributos de

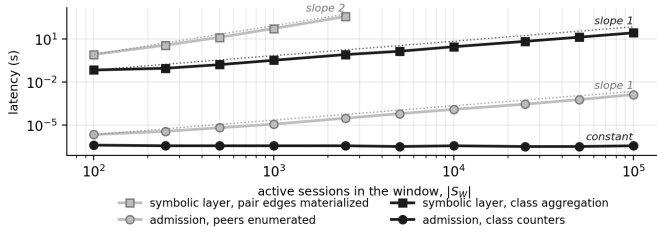


Fig. 3. Custo das duas camadas, com arestas entre pares e com contagens de classe.

fluxo) chega a 0,900 (AUC de 0,987), e a representação entre sessões, a 0,911 (AUC de 0,982), porque o Slowloris real, ao contrário das nossas campanhas furtivas, deixa uma assinatura de fluxo por sessão. A comparação com o KLAGÉ é favorável, mas *não controlada*. Nossa linha de base forte também supera o 0,841 publicado, de modo que a margem não se deve à representação. Além disso, o código disponibilizado não inclui pesos treinados e parte de um grafo cuja construção não foi publicada, o que impede uma reexecução controlada.

**Representação aprendida.** A configuração (d) é treinada e testada numa divisão 70/30 das sessões de uma mesma campanha. Treinada em 25 pilhas e testada em 100, ela alcança AUC de 0,75, contra 0,95 a 0,995 quando treinada no mesmo  $M$  com outras sementes. O gerador sorteia todas as sessões benignas antes de qualquer coisa que dependa de  $M$ , e por isso os testes entre valores de  $M$  treinam e testam em metades disjuntas das sementes. Numa divisão com a mesma semente, a configuração (a) alcançaria AUC de 0,88 a 0,94 por memorizar cópias das sessões.

**Validação da explicação.** As cadeias de evidência dos sete agrupamentos de laboratório passam nas verificações de completude (enumeram toda sessão correlacionada e toda sub-relação ativada) e de acionabilidade (o escopo é coerente com o discriminador do agrupamento), com 7/7, quando os agrupamentos são contados em sessões. Contados em origens, os cinco agrupamentos do CICIDS2017 têm de uma a três origens, abaixo de  $k_{\min}$ , e os dois do CIC-IoT2023 têm sete. Nenhum estudo com usuários testou essas cadeias.

## APÊNDICE D

### MODELO DE CUSTO E SENSIBILIDADE À JANELA

A admissão ocorre a cada requisição, enquanto a agregação de  $\Omega$  roda uma vez por janela. Por isso cronometramos as duas camadas separadamente, sobre uma janela sintética na mistura realista (30% das sessões numa campanha que compartilhe um JA4 de *botnet*, JA4 benignos tirados de um conjunto de 2000, oito *endpoints*, /24 dispersos), com medianas de três repetições em um núcleo. Cada camada roda de dois modos: com arestas entre pares, como as regras estão escritas, e com contagens de classe para as sub-relações baseadas em igualdade. Os dois modos fornecem o mesmo  $\Omega(S)$  para todo *endpoint*, com diferença de até  $7 \times 10^{-12}$ . A camada simbólica roda sobre *rdflib*, no lugar do Jena/TDB2, de modo que os números dessa camada são um limite superior. Além disso,

```
"@type": "kg:CoordinatedHTTFFlood",
"kg:clusterSize": 1991, "kg:coordinationScore": 1303422.8,
"kg:activatedSubRelations": [
{"@type": "kg:relatedByTLSPFingerprint",
"kg:coordinationWeight": 1.0, "kg:linkedPairs": 114710,
"kg:weightedContribution": 114710.0},
{"@type": "kg:relatedByEndpointConvergence",
"kg:coordinationWeight": 0.6, "kg:linkedPairs": 1981045,
"kg:weightedContribution": 1188627.0},
{"@type": "kg:relatedByNetworkProximity",
"kg:coordinationWeight": 0.3, "kg:linkedPairs": 286,
"kg:weightedContribution": 85.8}],
"kg:derivedMitigationScope": {
"kg:endpoint": "10.0.0.1:443",
"kg:tlsJa4": ["t13d1516h2_synth_0001_00", ...]}
```

Listagem 2. Cadeia de evidência de uma campanha canônica ( $M = 25$ ,  $K = 1000$ ), contada em origens. O escopo lista 25 impressões digitais (lista abreviada).

contar origens acrescenta uma consulta a uma tabela *hash* por admissão.

Com enumeração de pares, a latência mediana de admissão cresce de  $2,3 \mu s$  com  $|S_W| = 100$  para  $127 \mu s$  com  $|S_W| = 10000$ , acompanhando o número de pares candidatos (de 148 a 6495 arestas por admissão), à razão de 15 a 21 ns por par. Com contadores de classe, a latência fica entre 0,33 e  $0,42 \mu s$  de 100 a 100000 sessões (Fig. 3, em que as linhas pontilhadas são inclinações de referência). Materializar arestas entre pares custa 0,84 s com  $|S_W| = 100$  e 52,8 s com 1000, entre 198 e  $309 \mu s$  por aresta RDF. Esse caminho é linear no número de arestas e quadrático no de sessões. A agregação por classes, com uma tripla de pertinência para cada par (sessão, sub-relação) e com a consulta da Listagem 1, leva 0,33 s para 1000 sessões, 161 vezes mais rápido, e 26,4 s para 100000. O termo quadrático fica, assim, no valor de  $\Omega(S)$ , que conta pares, e nas arestas das sub-relações não transitivas, cujos pares candidatos são restringidos pelos índices. A cadeia de evidência (Listagem 2) precisa apenas das contagens de pares, que as classes fornecem.

No gerador,  $W$  é o intervalo que divide as sessões de um *endpoint* em agrupamentos de detecção. Uma varredura de  $W = 60$  a 1800 s sobre os mesmos cenários ( $K = 1000$ ,  $n = 10$ ) mantém a AUC de (d) em 0,976–0,978 e a de (c) em 0,663–0,664. O agrupamento da campanha tem de 1977 a 1998 sessões em qualquer  $W$ , enquanto o agrupamento médio cresce de 132 para 867 sessões, e o atributo discriminativo é a *fração* desse agrupamento que compartilha um JA4. A insensibilidade é, portanto, uma propriedade do gerador.

### Particionamento em Escala de CDN

A condição (ii), um único *endpoint* por  $S$ , faz do *endpoint* a chave natural de particionamento. Com essa partição, a regra é avaliada de forma *exata*, sem tráfego entre partições e com estado por partição limitado pela janela. Uma campanha que espalha o mesmo JA4 por muitos *endpoints*, por outro lado, não acumula, em nenhuma partição, pares suficientes para ultrapassar  $\tau_{\text{cluster}}$ , e particionar por JA4 quebra a condição de *endpoint*. Nenhuma chave isolada preserva as seis sub-relações.

Tabela V  
A REGRA POR JANELA DE CINCO MINUTOS, 30 EXECUÇÕES POR LINHA.

| Janelas            | $n_W$ | $\Omega \geq \tau$ | $\Omega$ e enriquecimento |       |      |
|--------------------|-------|--------------------|---------------------------|-------|------|
|                    |       |                    | Dispara                   | Cob.  | Col. |
| Ataque, $K = 1000$ | 90    | 77,8%              | 77,8%                     | 79,8% | 0,0% |
| Ataque, $K = 50$   | 35    | 85,7%              | 85,7%                     | 75,3% | 0,0% |
| Limpas             | 360   | 0,6%               | 0,0%                      | —     | —    |
| Flash crowd, 25    | 30    | 80,0%              | 0,0%                      | —     | —    |
| Flash crowd, 50    | 30    | 100%               | 0,0%                      | —     | —    |
| Flash crowd, 100   | 30    | 100%               | 0,0%                      | —     | —    |

Cob.: fração mediana dos atacantes da janela que o escopo bloqueia. Col.: maior fração bloqueada das sessões legítimas da janela.

A arquitetura que isso sugere, e que não implementamos, tem dois níveis: *workers* particionados por *endpoint* e um resumo global (*sketch*) de identificadores de peso alto, para recuperar campanhas que se espalham por vários *endpoints*. A revocação dessa arquitetura precisa ser medida antes de uma implantação em escala de CDN. Com a contagem por classes, as duas camadas crescem linearmente com a janela, e a admissão custa bem menos de um microssegundo.

#### APÊNDICE E A REGRA POR JANELA

Aqui a regra roda como seria implantada, por janela de  $W = 300$  s e por *endpoint*, e nenhum limiar é ajustado com ataques ou rótulos. O limiar  $\tau_{\text{cluster}}$  é o percentil 99 de  $\Omega$  sobre as janelas de 30 execuções livres de ataque, que também formam o perfil. Os falsos alarmes são contados em outras 30 execuções, e a detecção, nas campanhas canônicas ( $\alpha = 1,5$ ,  $M = 25$ ), sobre janelas com ao menos  $k_{\min} = 5$  atacantes. Um pico legítimo reposiciona 25, 50 ou 100 sessões benignas de uma terceira execução numa mesma janela. Espalhamos os inícios das sessões benignas por 60 min, porque o gerador os concentra nos primeiros minutos.

Um pico legítimo acrescenta massa da mesma forma que uma campanha. O teste distingue os dois porque esses picos sorteiam seus usuários da mesma mistura benigna que o perfil descreve, enquanto um surto concentrado num único tipo de cliente fica enriquecido, como mostram as frotas de produção. Com o teste, a regra não dispara em nenhuma janela limpa (limite superior do intervalo de confiança de 95%: 0,8%) nem em nenhum pico legítimo. Em toda janela de ataque que  $\Omega$  sinaliza, ela bloqueia, em mediana, de 75 a 80% dos atacantes, sem atingir nenhuma sessão legítima. Com pesos uniformes,  $\Omega$  detecta pelo menos tantas janelas de ataque quanto com os pesos propostos (78,9% e 85,7%), e, sem o termo de *endpoint*, detecta 33,3% das janelas de ataque com  $K = 1000$  e nenhuma com  $K = 50$ , embora ainda sinalize 53 a 100% dos picos legítimos. Um gatilho de origens distintas no percentil 99 captura 82,2% e 85,7% e, com o teste, também não dispara em nenhuma janela limpa nem em nenhum pico legítimo.

**Tráfego de produção.** As exportações vêm dos quatro *endpoints* de serviços próprios da operadora cujas janelas têm ao menos  $k_{\min}$  origens. O tráfego limpo exclui os clientes bloqueados pelo WAF, e, como as exportações não trazem

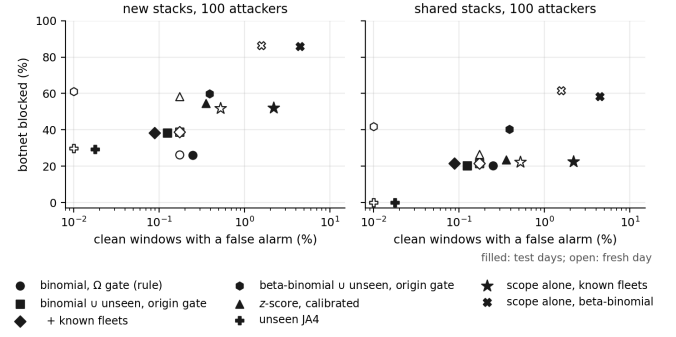


Fig. 4. Pontos de operação no tráfego de produção, 100 atacantes por janela (uma taxa nula é desenhada em 0,01%).

prefixos, o escopo é julgado apenas pelo JA4. A *botnet* injetada põe 90% dos atacantes em 25 pilhas, com 1/24 tirados de 2000 prefixos disjuntos dos reais. Frotas legítimas, como sondas ativas só em horário comercial, levam  $\lambda_e$  aos níveis da Tabela IV, e o escopo sozinho produz então um filtro em 1,5 a 6,3% das janelas do dia seguinte. Quando a regra dispara numa janela limpa de E2 ou E3, ela aponta uma frota e bloqueia cerca de metade dos clientes dessa janela. O gatilho de origens distintas reduz esses falsos alarmes à metade. As oito janelas limpas em que o escopo produz um filtro e que só o gatilho  $\Omega$  admite têm 33% do seu  $\Omega$  fora do termo de *endpoint*, contra 12% nas demais, porque uma frota se concentra numa impressão digital e eleva o termo TLS.

**Concordância com o WAF.** Dos clientes que o escopo bloquearia no tráfego completo, o WAF também bloqueou 84% e 96% nos dois *endpoints* em que o escopo não aponta frotas, e 1,5% naquele em que aponta. O escopo cobre 16,7% de todos os clientes que o WAF bloqueou.

**Outras linhas de base.** Sem calibração, o  $z$ -score dispara em 2,6% das janelas limpas e em 64,5% dos picos legítimos, e um  $z$ -score de cada impressão digital em relação ao próprio histórico, em metade desses picos.

**Referência sobredispersa.** A variante beta-binomial atribui a cada impressão digital do perfil uma correlação dentro da janela,  $\phi$ , estimada pelo método dos momentos sobre as janelas de calibração a partir de  $\text{Var}[c] = nb(1-b)(1+(n-1)\phi)$  e limitada ao intervalo  $[0; 0,99]$ . A contagem de cada impressão digital é então testada em relação a uma beta-binomial de mesma média. Uma impressão digital ausente do perfil continua testada pela binomial, e o nível é calibrado como antes.

Dos quatro limiares testados para declarar uma frota conhecida (fração das janelas de calibração em que o teste aponta), só o de 5% não aumentou os falsos alarmes nos três primeiros dias de teste. Nos dois últimos, esse limiar eleva, em E1, a fração de atacantes que o enriquecimento bloqueia numa *botnet* de um décimo da janela típica, de 0,7% para 6,3%, com 7 falsos alarmes contra 8, e deixa E2 a E4 inalterados. Limiares menores bloqueiam mais atacantes, mas disparam com o dobro da frequência, com filtros que bloqueiam menos clientes (mediana de 11 a 12% dos clientes, contra 51%).

O piso da Tabela IV é o tamanho da menor *botnet* cujas pilhas atingem, na janela mediana do *endpoint*, a contagem mínima para que o teste, nesse nível, as aponte. As linhas de base calibradas tomam como limiar o percentil 99 do maior  $z$  sobre as janelas de calibração, nunca abaixo de 3. Um gatilho acionado apenas pelo escopo dispara, com as frotas conhecidas isentas, em 2,2% das janelas limpas de todos os *endpoints* e em 0,5% no dia novo, contra 3,5% e 8,8% sem esse tratamento (Fig. 4). Um perfil por hora do dia não ajudou. Usar  $\rho = 2$  ou 5 não muda nenhuma taxa agregada de disparo em mais de 0,3 ponto percentual nem nenhuma fração bloqueada em mais de 4,5 pontos, enquanto uma mediana de dano colateral chega a mudar 7,2 pontos. Picos legítimos de 1 000 usuários disparam a regra em 21,8% das janelas, e o  $z$ -score, em 86,8%.

#### APÊNDICE F REPRODUTIBILIDADE

A implementação de referência está disponível em <https://github.com/NatanaelOliveira/knowledge-graph-ddos-article>, sob licença permissiva. O comando `make compile-check stix-ingest` confere em sessões geradas a consulta de contagens compilada e submete os pacotes exportados ao validador STIX 2.1 da OASIS em modo estrito, ao TAXII 2.1 e ao importador do MISP. O protocolo do dia novo foi registrado, com os *hashes* de suas exportações, antes de o dia ser lido (`fresh_day_protocol.md`), e o comando `make production-tables` recalcula todos os números de produção, com os dias de teste e o dia novo separados. As duas derivações de escopo são distribuídas lado a lado, para que o resultado negativo da Seção V-C possa ser reproduzido, e não apenas afirmado. Os rótulos só entram na avaliação: perfis e níveis vêm de execuções livres de ataque, e os scripts que geram a Tabela II tomam o agrupamento de maior  $\Omega$  sem rótulo algum. As sementes são fixas, e Makefiles conduzem cada estágio, de modo que toda tabela e todo gráfico se regeneram com um único comando. A exceção são as entradas das Tabelas III e IV, agregados da operadora, cujo resumo anonimizado acompanha o artefato e é conferido com as tabelas. A ontologia OWL é distribuída em RDF/XML com suas anotações `coordinationWeight`, e pretendemos submeter os artefatos à trilha de Avaliação de Artefatos.